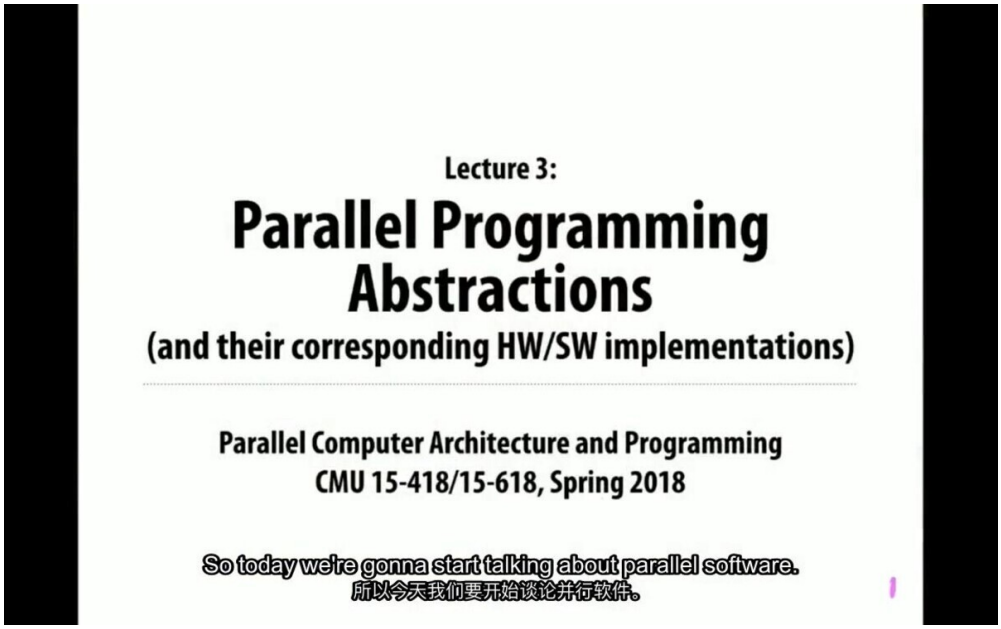
课程笔记

并行编程抽象：抽象与实现的差距

CMU 15-418/618 并行计算机架构与编程 · Lecture 3（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 7 分 38 秒
视频链接：`https://www.bilibili.com/video/BV1G94y1k7zt/`

目录

1 抽象与实现：并行编程最容易混淆的一对概念 3

1.1 为什么这两个概念必须分开 . . . 3
1.2 本讲的两条主线 . . . 3
1.3 本章小结 . . . 3

2 ISPC：看起来像多线程，实际是向量指令 3

2.1 SPMD 抽象的定义 . . . 4
2.2 ISPC 的代码组织 . . . 4
2.3 programCount 与 programIndex . . . 4
2.4 一个必须避免的误解 . . . 5
2.5 uniform 提示 . . . 5
2.6 gap 大小的真相 . . . 5
2.7 跨核扩展：task . . . 6
2.8 本章小结 . . . 7

3 两种数据分布：interleaved 与 blocked 7

3.1 交叉分布 interleaved . . . 7
3.2 阻塞分布 blocked . . . 8
3.3 两种分布各自的优缺点 . . . 9
3.4 ISPC 中的真正答案 . . . 9
3.5 让编译器决定：forEach . . . 11
3.6 本章小结 . . . 12

4 归约：为什么求和不能标 uniform 12

4.1 失败的尝试 . . . 12
4.2 正确的做法：部分和加 reduce . . . 12
4.3 本章小结 . . . 13

5 其他抽象及其实现层面的差异 13

5.1 系统分层：抽象在哪，实现在哪 . . . 13
5.2 pthreads：库与内核共同承担 . . . 14
5.3 共享地址空间抽象 . . . 15
5.4 共享地址空间的硬件实现 . . . 15
5.5 NUMA：放弃等距 . . . 17
5.6 本章小结 . . . 18

6 消息传递：彻底放弃共享内存 18

6.1 消息传递模型的通信方式 . . . 18
6.2 本章小结 . . . 19

7 数据并行：把并行交给编译器 19

7.1 对全部数据应用同一函数 . . . 19
7.2 forEach 只保证控制并行 . . . 20

7.3 stream：用纯函数避免竞争 20
7.4 本章小结 20

8 总结与延伸 **20**

1 抽象与实现：并行编程最容易混淆的一对概念

这一讲要解决的问题很简单，却几乎人人都会在这一步出错：当你说”线程”时，你说的是语言的抽象，还是硬件里真的有一个线程？

1.1 为什么这两个概念必须分开

抽象是为人服务的，它让程序更容易表达、更容易调试。但抽象终归要被实现。并行领域特别容易把两者混淆，因为抽象和实现都在处理同一件事——并发与通信。语言用抽象表达并发，系统也在管理并发，两者用着相似的名词，含义却完全不同。

讲者把这一讲的题目定为”抽象与实现的区别”，并选择 Intel 的 ISPC 语言作为第一个案例，因为这个语言把反差做到了极致。

1.2 本讲的两条主线

课程后续的所有内容都可以归到一组对照上：程序员看到的模型是什么，机器实际做的事情是什么。本讲后面 ISPC 讨论部分的一张幻灯片把这对关系写成了两行，如下图所示。

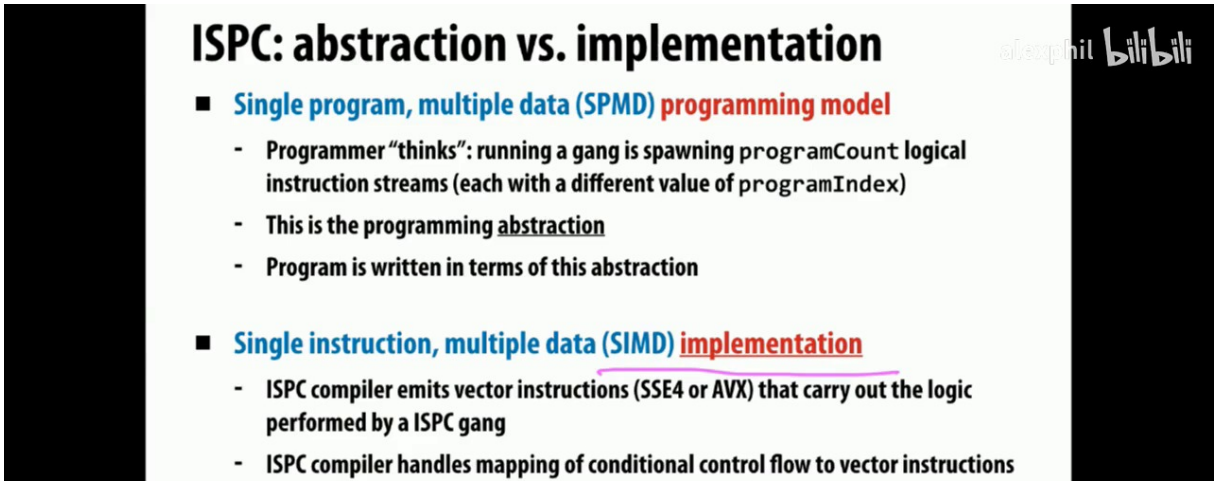


图 1: ISPC 的抽象与实现对照：抽象是 SPMD 编程模型，实现是单线程上的 SIMD 指令[1]

第二主线是通信。并行与串行真正不同之处，在于并发执行的多份代码必须相互通信、相互配合。因此每个抽象都要回答同一个问题：你用什么手段来表达通信。

1.3 本章小结

抽象是人造的便利，实现是机器的真相，并行领域因为名词重合而特别容易混为一谈。本讲用 ISPC 演示最尖锐的一例——看起来是多线程，实际是单线程加向量指令；随后讨论另外三个主流抽象，并分别指出它们的实现方式与代价。

2 ISPC：看起来像多线程，实际是向量指令

ISPC 是这一讲的核心案例。理解它，就能看清抽象与实现能差多远。

[1] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

2.1 SPMD 抽象的定义

ISPC 的核心抽象叫 SPMD，即单程序多数据。它的含义是：有多个并发执行体，每个执行体运行同一段代码，但各自处理不同的数据。

讲者特别解释了为什么他尽量回避” 线程” 这个词。抽象里说的是逻辑上并发的实例，它们可能同一时刻执行，也可能不是严格同步；关键在于给它们同一段代码，让它们各自在自己的数据上执行。

与之相对的是另一种风格：完全不同的程序处理不同的数据。浏览器这类软件就是这样——多个互不相关的任务各做各的事，执行的是完全不同的过程。但在这门课里，目标是把同一件事做快，所以采用同一份代码的模式。

2.2 ISPC 的代码组织

ISPC 的模型是受限的、明确的：把需要并行执行的部分抽出来，放进独立的`.ispc` 文件。这些文件的代码看起来很像 C，只是扩展名不同。文件之外是调用它们的普通代码，例如 `main.cpp`，顺序执行，直到调用到`.ispc` 里的方法才进入并行。

ISPC 不用” 线程” 这个说法，而用 gang（一帮、一组）和 program instance（程序实例）。这个措辞刻意模糊，因为抽象层级上还没有落定到具体实现。

执行过程因此呈现三段：调用前顺序执行，进入 ISPC 方法时并发执行，返回后回到顺序执行。所有的并行都通过创建这些 ISPC 文件来表达。

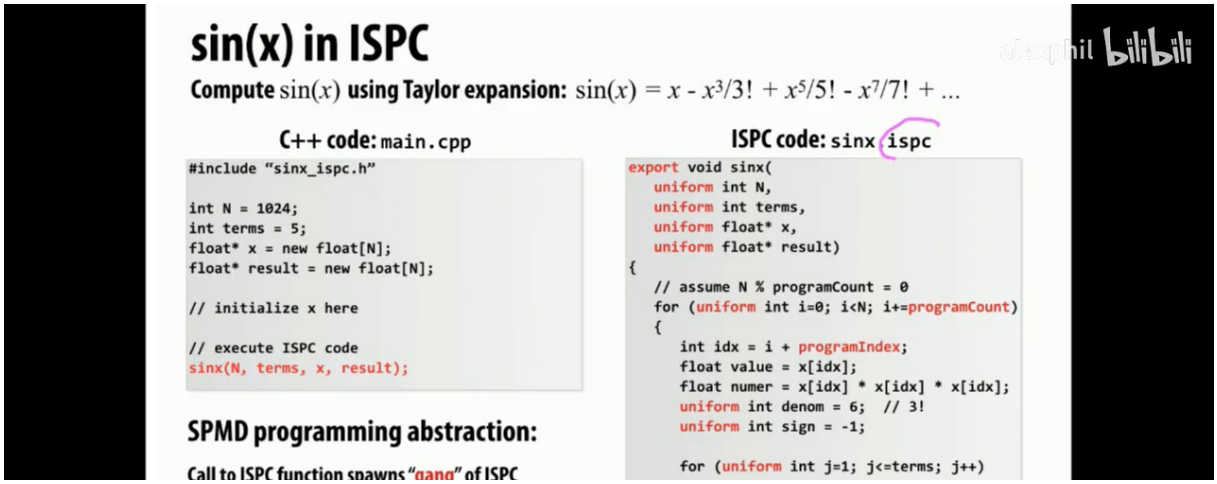


图 2: ISPC 版本的 sin(x)：C++ 侧保留顺序代码，并行部分放在.ispc 文件[2]

2.3 programCount 与 programIndex

ISPC 文件里有两个关键的内置变量。programCount 是这个 gang 中并发实例的数量，也就是同时有多少件事在并行。programIndex 则是” 我是第几个实例”，取值从 0 到 $n-1$。

programCount 有一个不寻常之处：程序员不能设置它。它由运行时系统决定。程序里只能读取。

利用这两个变量，原本的顺序循环被改写成跳跃式遍历：

[2]视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

```
// 在 .ispc 文件中
export void sinx(uniform int N, uniform int terms,
                 uniform float input[], uniform float result[]) {
    // 外层循环按 programCount 跳跃，而不是每次加 1
    foreach (i = 0 ... N) {
        // 每个实例计算自己的下标
        int index = i + programIndex;
        ...
    }
}
```

Listing 1: ISPC 用 programIndex 切分迭代（按讲者对两个内置变量的描述整理）

讲者解释了这个改写的作用：如果 programCount 是 4，外层循环一次前进 4，四个并发实例同时运行，各自的下标都由自己的 programIndex 决定，因此它们在做不同的工作。

2.4 一个必须避免的误解

如果只是把原来的 `sinx` 函数原封不动搬进 ISPC 文件并调用，会发生什么？讲者的回答是：它会创建 programCount 个并发实例，但每个实例做的工作完全相同，彼此完全冗余，因此程序不会变快。

把顺序代码搬进并行文件是最常见的空转

原样搬入 ISPC 文件后，programCount 个实例执行完全相同的工作，结果彼此覆盖，耗时不变。判断一份并行代码是否真的在工作，看的是各实例访问的下标是否互不相同，而不是文件扩展名。

这个反例值得记住。并行抽象本身不产生加速，真正产生加速的是把工作切分开。写并行代码的第一件事永远是划分数据，而不是套用并行语法。

2.5 uniform 提示

ISPC 还有一个修饰符叫 uniform。它标注的是：这个变量在所有并发实例中取值完全相同。

例如 index 就不是 uniform 的，因为每个实例根据 programIndex 和 i 算出的值不同；而参与计算的常量、数组长度这类量则是 uniform 的。

讲者说这是一个不寻常的提示——在多数并行抽象里没有这个概念。他承诺稍后揭开实现细节时，读者就会明白为什么需要它。

2.6 gap 大小的真相

到 ISPC 讨论部分，讲者揭开了底层实现。ISPC 是为一个明确目的设计的：让编写利用 SIMD 向量指令的代码变得容易。如果手写 intrinsics，要逐个指明做向量加、向量乘，相当麻烦。ISPC 的做法是让程序员写看起来普通的并发代码，由编译器在背后生成向量指令。

programCount 是实现的函数，不是程序员的参数

programCount 由运行时按机器与数据类型算出来，程序员只能读。这意味着同一份 ISPC 源码在 256 位与 512 位机器上会有不同的并发实例数，循环的跳跃步长随之改变。写 ISPC 时不能假设某个固定的 gang 大小。

因此那个不¨ 寻常的¨ 答案就出来了：

$$\texttt{programCount} = \frac{\text{向量宽度}}{\text{单元素位宽}}$$

- programCount：gang 中并发实例数，由运行时决定；
- 向量宽度：机器 SIMD 寄存器宽度，例如 256 位；
- 单元素位宽：数据类型的位数，例如单精度浮点为 32 位。

对 256 位寄存器与 32 位单精度浮点，programCount 就是 8。所谓 gang 里并发的八个实例，实际上是向量寄存器中的八个通道。

讲者紧接着给出结论：ISPC 生成的不是多个线程，而是单一的线程执行流，并行完全来自 SIMD 向量指令。

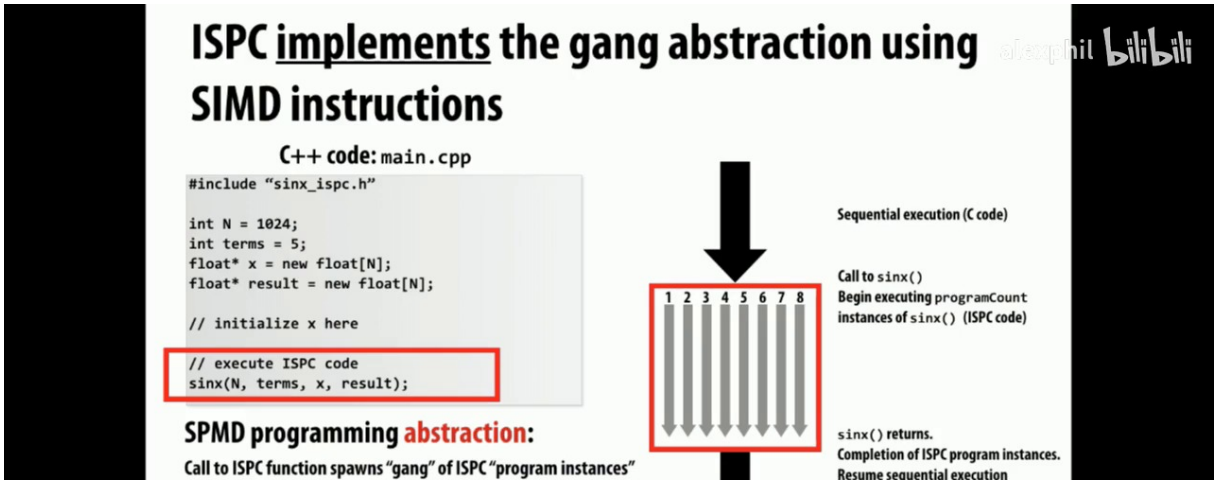


图 3: ISPC 用 SIMD 指令实现 gang 抽象[3]

2.7 跨核扩展：task

programCount 通常就是向量宽度，但机器往往有多个核心。前面所有代码只会跑在一个核上，用不到其余核心。

因此 ISPC 提供另一个机制：task。task 是可以在不同核心上运行的控制流，它会被转成真正的线程。

[3] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

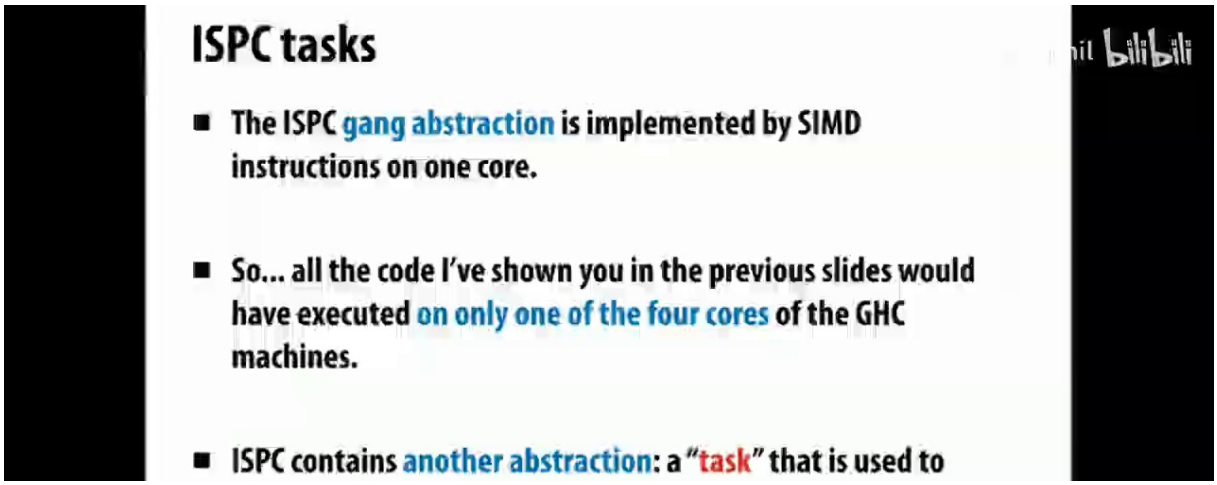


图 4: ISPC task：gang 抽象只用一个核，跨核需要显式创建 task[4]

2.8 本章小结

ISPC 的抽象是 SPMD 的多实例并发，实现是单线程加 SIMD 指令，programCount 等于向量宽度，uniform 提示正是为了让编译器能复用同一个向量寄存器。要跨越多个核心，必须显式使用 task。

这个案例确立了一条判断规则：看到”并发实例”时，先问它在硬件上对应什么。同样的抽象可以用完全不同的机制实现，性能特性也随之不同。

3 两种数据分布：interleaved 与 blocked

同样的 gang 抽象，把数据分配给实例的方式可以不同。这一节的结果出人意料：在 ISPC 上两者差距极大。

3.1 交叉分布 interleaved

前面看到的代码采用的是交叉分布：实例按 programIndex 轮流取数据元素。四个实例时，第 0 个实例处理下标 $0, 4, 8, 12 \ldots$，第 1 个处理 $1, 5, 9, 13 \ldots$，依此类推。

[4]视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

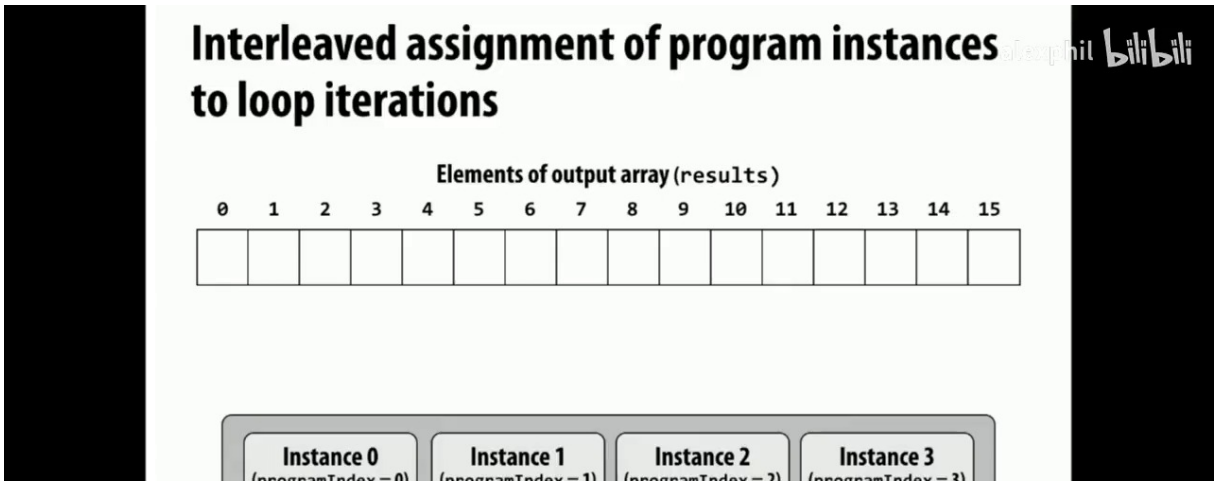


图 5: 交叉分布：程序实例与循环迭代的对应关系[5]

从某个实例的视角看，它跳着访问；但从整体看，任一时间点上四个实例正在处理连续的四个元素。这一点在后面决定了性能。

交叉分布的索引映射可以写成显式公式：

$$i_k = k + j \cdot P, \qquad k = 0, 1, \ldots, P-1$$

- i_k：第 k 个程序实例在第 j 轮迭代中处理的数据下标；
- k：programIndex，即实例编号；
- P：programCount，即 gang 中的并发实例数；
- j：外层循环的轮次。

3.2 阻塞分布 blocked

另一种做法是把数组切成连续的大块，每个实例负责一块。代码逻辑因此不同：先算出每块包含多少个连续元素，再算出自己从哪个位置开始。

阻塞分布的映射则是按块划分：

$$\text{chunk} = \left\lceil \frac{N}{P} \right\rceil, \qquad \text{start}_k = k \cdot \text{chunk}$$

- N：要处理的元素总数；
- P：程序实例数；
- chunk：每个实例负责的连续元素个数；
- start_k：第 k 个实例的起始下标。

讲者提醒暂时忽略 N 不能被 P 整除的情形，那属于边界处理而非模型本身。

[5] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

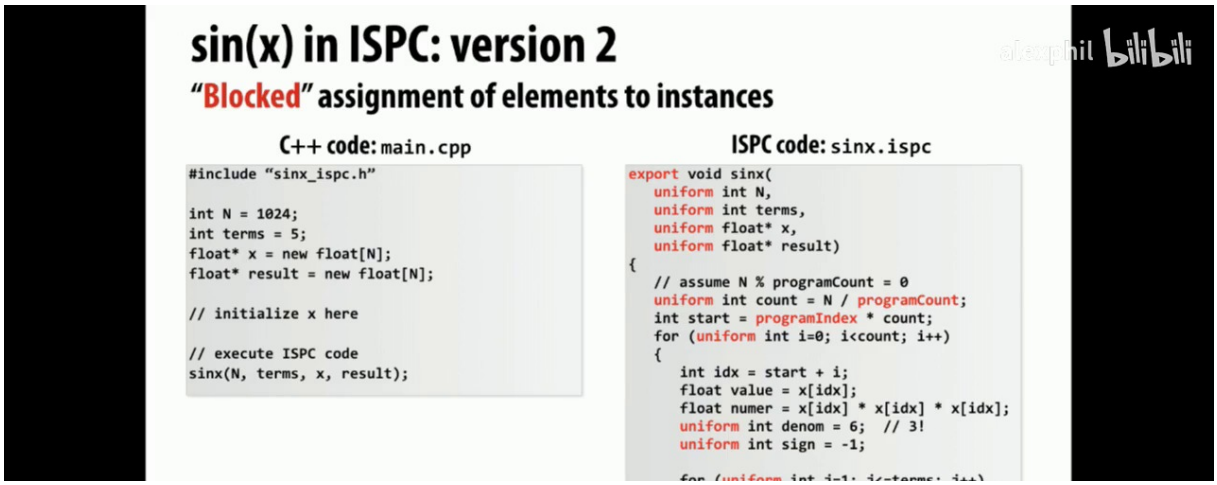


图 6: 阻塞分布的 ISPC 代码：先算块宽，再算起始位置[6]

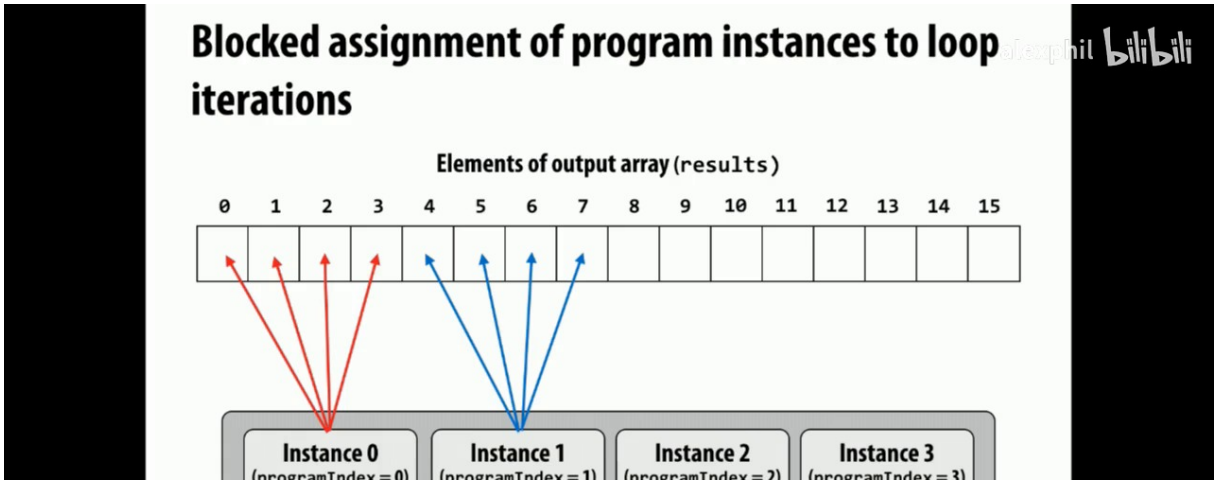


图 7: 阻塞分布下实例与迭代的对应关系[7]

3.3 两种分布各自的优缺点

讲者请学生比较两者，得到的是两个真实的权衡，但都不是最终答案。

阻塞分布的缺点是可能负载不均。如果计算时间随索引增长（例如随 n 的平方增长），那么均匀切块之后，左边那块任务很轻，右边那块任务很重。相比之下，交叉分布在计算量呈现连续变化时风险更小。

阻塞分布的优点是空间局部性好，这对缓存有利。如果多个线程各自在独立的缓存上工作，连续访问数据通常更快——这在 2 到 13 课程里是常识。

3.4 ISPC 中的真正答案

讲者随即指出，在 ISPC 上这两个分布有明显得多的胜负，而且原因不是上面任何一条。

[6] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[7] 视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

原因是时间视角。在交叉分布下，任一时间点向量指令处理的是连续的下标 0 到 3。当要把数据装进向量寄存器时，需要的是连续内存——而这正好是交叉分布提供的。有专门的向量加载指令可以一次搬入一大块连续数据，只要数据本身连续，效率极高：一次加载操作就取回全部需要的值。

阻塞分布则相反。时间上它需要同时加载下标 0、4、8、12，这些在内存中并不连续。硬件确实有向量 gather 指令能一次加载多个非连续元素，但代价远高于普通向量加载——它实际上相当于执行了四次或八次独立加载。从汇编程序员角度看这是一条指令，从性能角度看它非常昂贵。

其代价可以近似理解为：

$$T_{\text{gather}} \approx W \cdot T_{\text{load}}$$

- T_{gather}：一条 gather 指令的实际耗时；
- W：向量宽度，即一次装入的元素个数；
- T_{load}：一次普通标量加载的耗时。

这解释了为什么交叉分布在 ISPC 上胜出得如此干脆：它让向量加载始终能用到连续的 T_{load}，而阻塞分布只能反复付出 W 倍的代价。

抽象一致不代表性能一致

两种分布都能表达同一个 gang 抽象，都正确计算出结果，但在 SIMD 实现下一个比另一个快得多。性能差异来自实现层的数据搬运方式（连续向量加载对比 gather），与抽象层的语义无关。选择数据分布时，必须考虑目标实现，而不只是逻辑正确性。

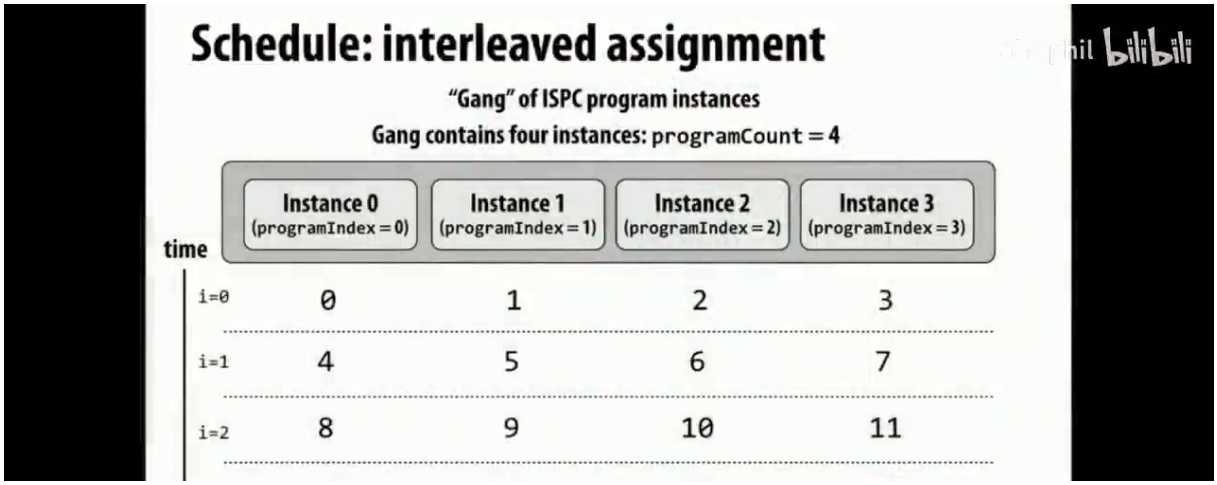


图 8: 时间轴上的交叉分布：同一时刻向量处理连续下标，可用单条向量加载[8]

[8] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

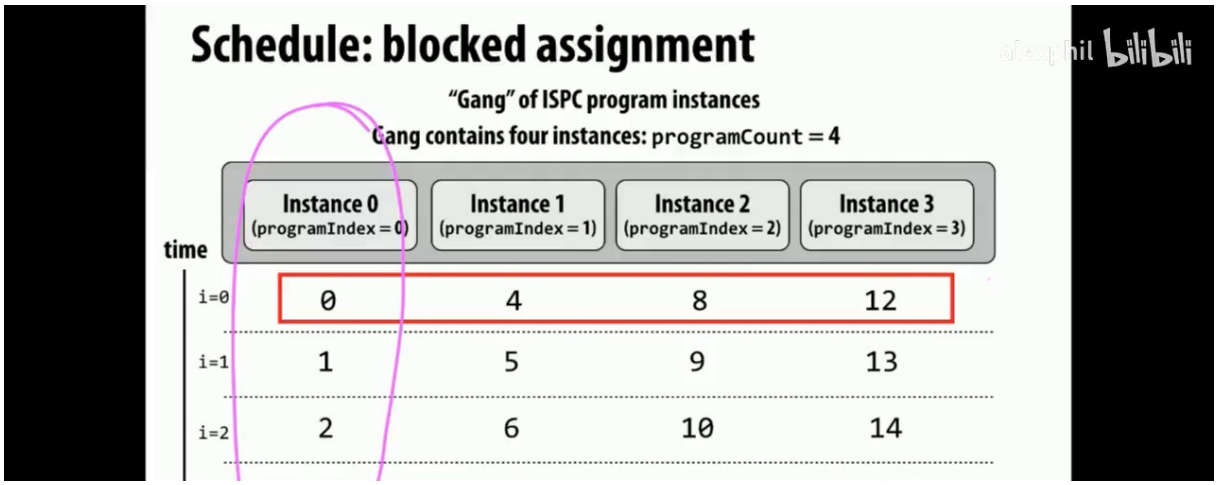


图 9: 时间轴上的阻塞分布：同一时刻需要跨步取数，只能依赖 gather[9]

3.5 让编译器决定：forEach

既然手工选择容易出错，ISPC 提供了 forEach：把 for 循环换成 forEach，等于告诉系统这些迭代可以并行、顺序由你决定。

forEach 交换的是确定性与可移植性

把迭代分配权交给系统，代码从此不绑定某一种数据分布，换实现时无需改写。代价是性能特征变得不那么直观：同一个 forEach 在不同编译器、不同目标架构下可能综合出不同的分布与访问模式。需要精确控制内存行为时，仍需退回手工分布。

系统会自行综合出代码，在 ISPC 里它选择交叉分布。

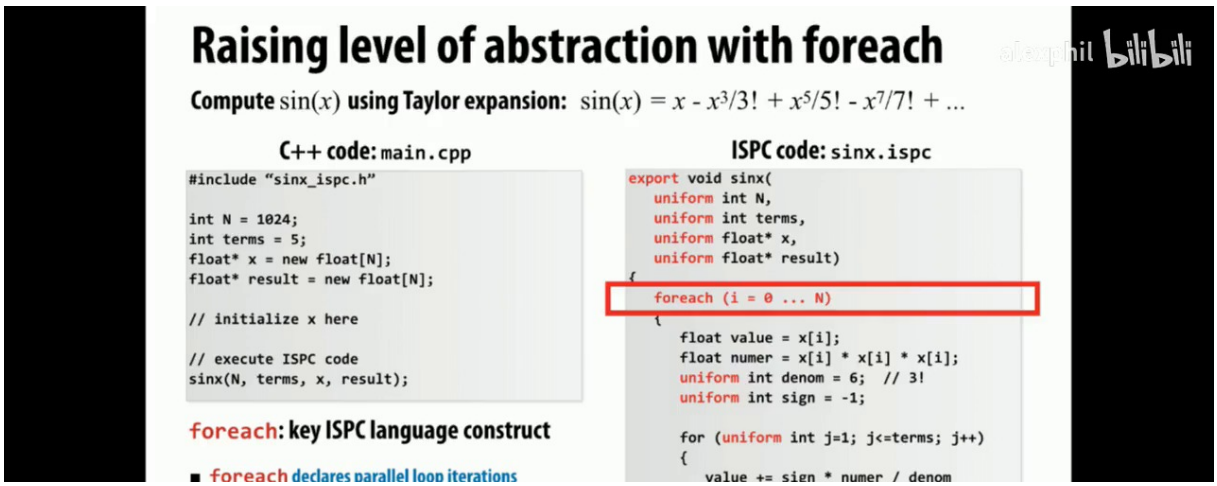


图 10: 用 forEach 提升抽象层次：交出迭代分配权，由系统综合[10]

[9] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。
[10] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

3.6 本章小结

交叉分布与阻塞分布各有负载均衡与空间局部性的权衡，但在 SIMD 实现下交叉分布压倒性胜出，因为向量加载要求同一时刻的数据连续，而阻塞分布只能走昂贵的 gather。这个例子说明抽象正确不等于性能正确。

顺带一个限定：讲者提醒这个结论依赖 ISPC 是单线程加向量指令；若换成各自独立缓存的真线程，阻塞分布的空间局部性优势就会显现。

4 归约：为什么求和不能标 uniform

前一节处理的是”每个输出独立”的情形。归约则相反：所有实例共同产生一个结果。这一节看清了这个转变带来的约束。

4.1 失败的尝试

任务是把数组所有元素求和并返回。讲者提出一个自然的想法：既然最终大家应该得到同一个和，那就在 sum 声明前加 uniform。

ISPC 会直接报编译错误。

uniform 描述的是取值，不是逻辑意图

代码意图上本轮归约的最终结果确实只有一个值，但 uniform 修饰的是每个实例此刻持有的变量值。各实例累加的 x_i 不同，因此被修饰的变量在运行期间不 uniform。判断能否标 uniform，看的是运行时的取值分布，而不是作者期望的结果。

原因在数据依赖：每个实例要累加的是不同的 x_i，这些值本身不 uniform。如果让多个实例并发地往同一个变量里加，朴素实现会出错——这正是第 5 章要讨论的共享变量竞争问题在向量语境下的重现。

4.2 正确的做法：部分和加 reduce

可行的写法分两步。先让每个实例算出自己那部分的部分和，这些部分和确实每个实例不同、不 uniform。然后用 ISPC 的 reduce_add 原语把这些不同的值合并成最终结果。

归约在形式上就是把各实例的局部结果合并为单一值：

$$S = \sum_{k=0}^{P-1} S_k, \qquad S_k = \sum_{i \in I_k} x_i$$

- S_k：第 k 个实例算出的部分和，逐实例不同，因此不 uniform；
- I_k：第 k 个实例负责的下标集合；
- S：合并后的最终结果，全部实例共享同一值，可以标 uniform；
- P：程序实例数。

讲者展示了生成代码的样子：前半段是向量指令的加、加载、存储；后半段做归约时反而不用向量指令，而是用常规指令把向量的各元素逐个加起来。

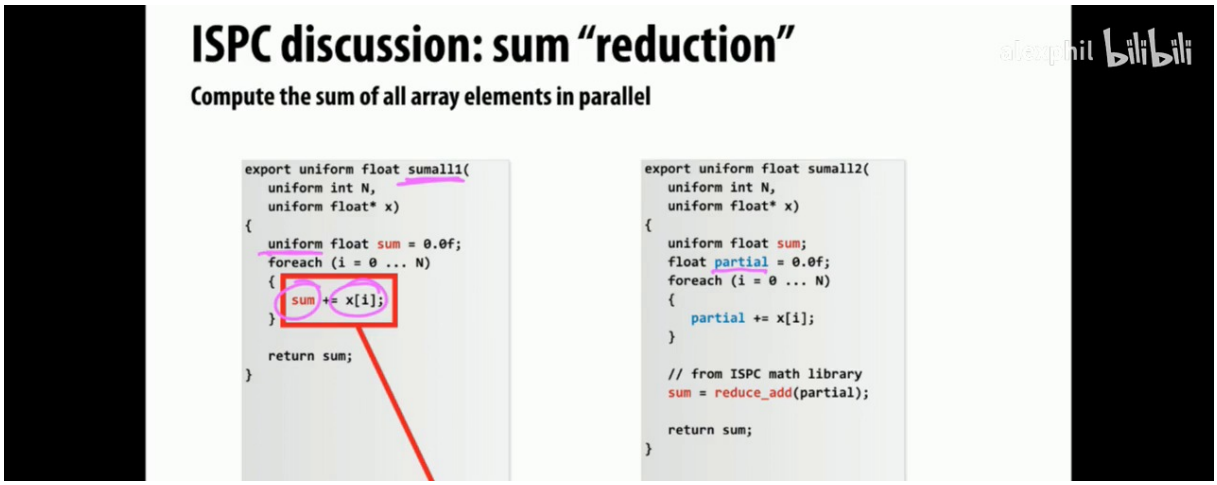


图 11：求和归约：先算部分和，再用 reduce_add 合并[11]

归约的代价藏在最后一步

归约把并行度收拢成一个标量，这一步无法向量化，只能用标量指令逐个累加。因此归约不是免费的：它把前面并行获得的收益按元素数取对数或线性地吃掉一部分。频繁的小规模归约往往是并行程序的隐性瓶颈。

4.3 本章小结

归约破坏了” 每个输出独立” 的前提，因此不能声明为 uniform，必须显式用部分和加 reduce 原语，且最后一步退化为标量运算。至此 ISPC 的抽象、实现、数据分布与归约四条线索都已交代清楚。

接下来要问的是：ISPC 只是众多抽象之一，还有哪些主流抽象，它们的实现方式又差多远?

5 其他抽象及其实现层面的差异

ISPC 只是入口。这一节把视角拉远，比较三种主流并行抽象以及它们各自落在软件栈的哪一层。

5.1 系统分层：抽象在哪，实现在哪

讲者先建立了一幅分层的图景。程序员写高级语言，其下是编译器、运行时与库，再下面是操作系统，最下面是硬件。

抽象可以在软件实现、硬件实现，或者两者结合。而且” 软件” 本身也不是单一的一层。这个框架是判断性能来源的工具。

[11] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

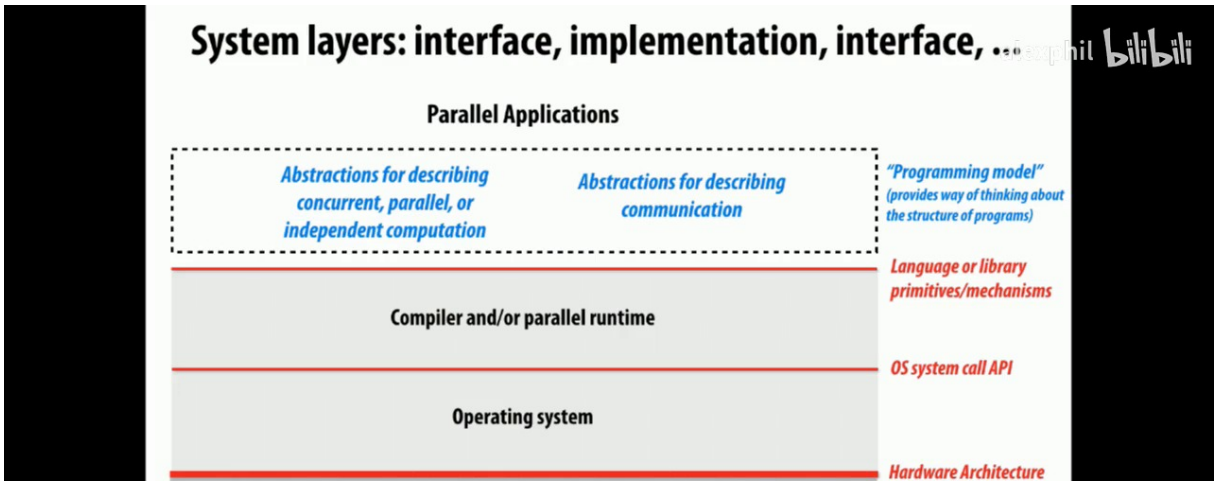


图 12: 系统分层：应用、抽象、编译运行时、操作系统与硬件[12]

5.2 pthreads：库与内核共同承担

创建线程调用 pthread_create，这是一个库函数。它通常需要内核支持——如果希望线程跑在另一个核心上，内核必须知道这个线程存在。而操作系统本来就在管理多个核心上的执行，也已经在与硬件交互。

因此 pthreads 的大部分工作发生在库层与内核层。

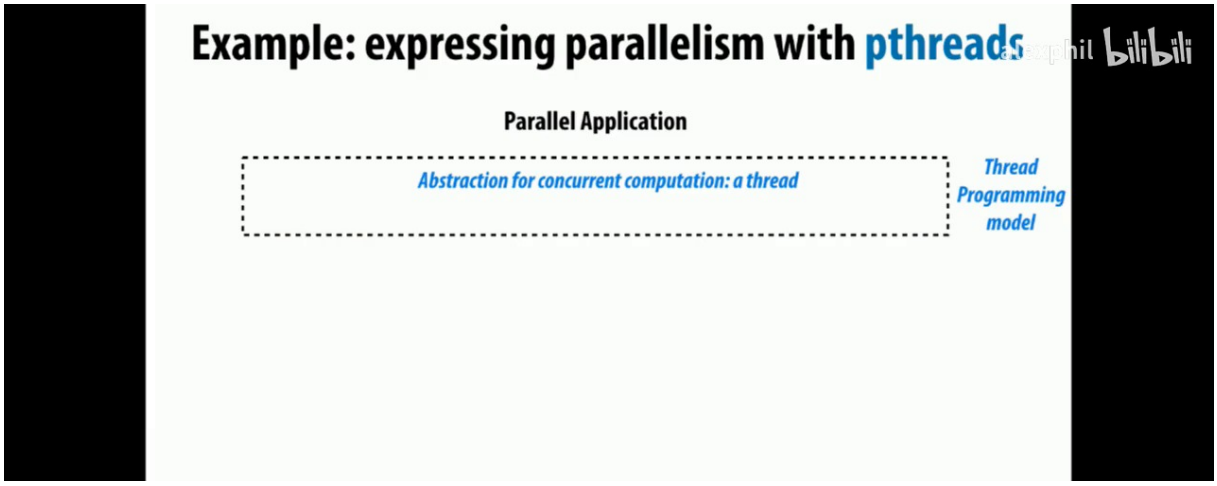


图 13: 用 pthreads 表达并行：抽象落在库与内核层[13]

对比之下，ISPC 通常完全不涉及操作系统：编译器只生成向量指令，硬件直接并行执行，不需要特殊库或内核参与。繁重的工作全部由编译器承担。

[12] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

抽象	主要实现层	是否需要内核	并行单位
ISPC gang	编译器	否	向量通道
ISPC task	编译器 + 运行时	是（转成线程）	线程
pthreads	库 + 内核	是	线程
共享地址空间	硬件 + 缓存一致性协议	否（硬件内建）	线程
消息传递	网络硬件 + 库	否（只需网络）	进程

表 1: 四种抽象的实现层次对照

5.3 共享地址空间抽象

这是最常见的模型：所有并发执行体共享同一个地址空间，通过读写共享变量来通信，额外要做的是加锁与其他同步机制。

讲者指出，将来看到这类代码时会发现，多出来的代码大多是为了同步原语。他也用一个例子预告了危险：若两个线程都读出同一个值、各自加一、再写回，就可能丢掉一次自增。

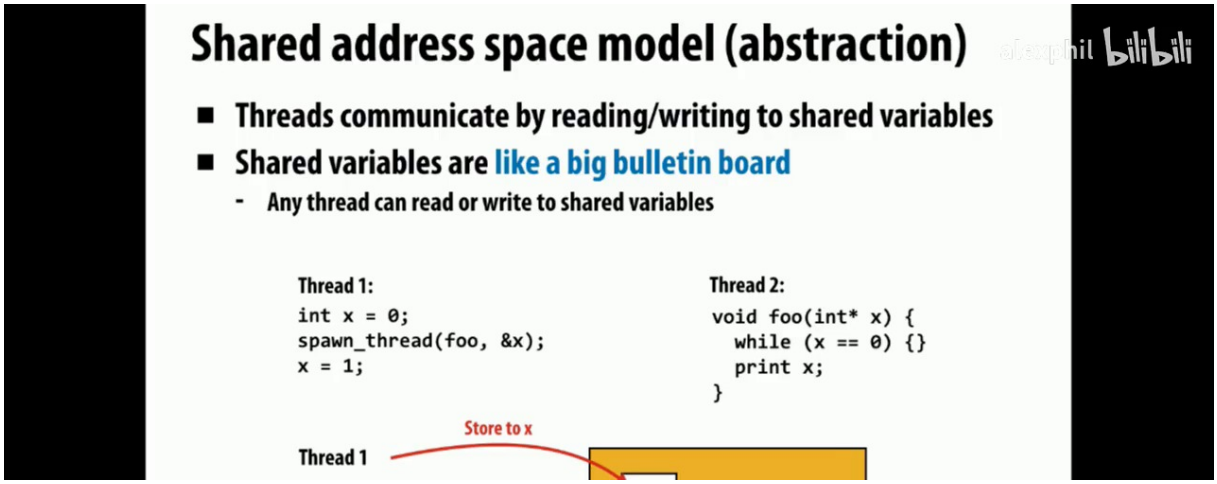


图 14: 共享地址空间模型：线程通过读写共享变量通信[14]

5.4 共享地址空间的硬件实现

程序员视角的简单，在硬件上要付出代价。最直接的做法是让处理器物理上共享同一份内存，通过总线或其他互连连接。

这种结构有个别名叫 dance hall 架构——像中学舞会，所有人都贴在墙边，没人站在中间；处理器在一边，内存在另一边。它也被称为对称多处理：每个处理器到内存的距离相等。讲者的评语是，这听起来不错，但实际上是” 对所有人一样糟”。

[14] 视频画面时间区间：00:32:30–00:32:45。若为裁剪图，时间区间仍对应原视频画面。

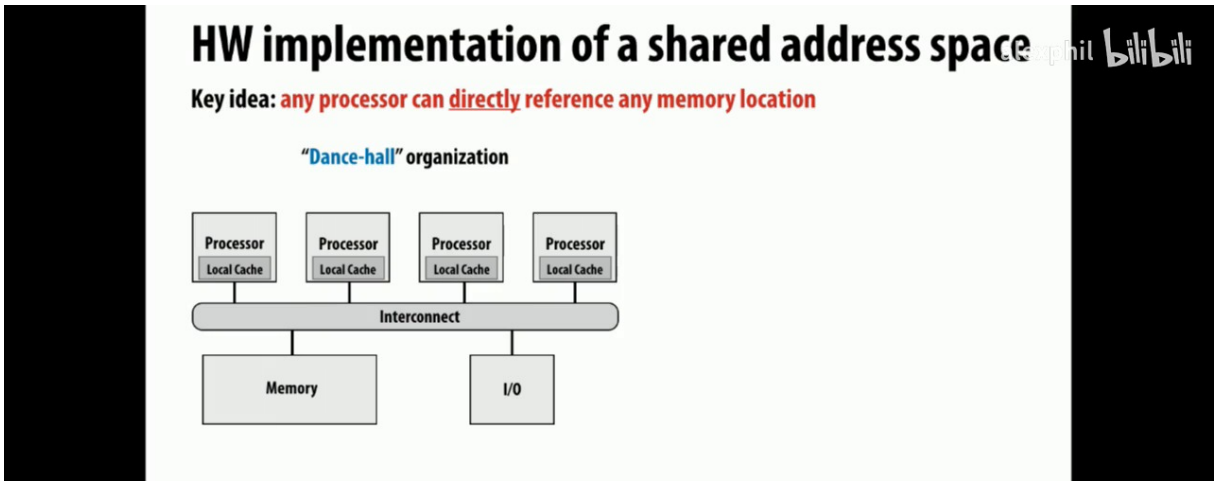


图 15: 共享地址空间的硬件实现：任一处理器都能直接引用任一内存地址[15]

问题在互连。总线很便宜，但同一时刻只能有一个设备在总线上，带宽无法扩展。要用更复杂的方式连接更多处理器与更多内存模块，成本迅速上升。

真实硬件的例子可以看出这个压力。现代处理器上若干核心共享同一个缓存，这是最贴近对称多处理的做法。而 Sun Niagara 2 为了支持大量容忍延迟的任务，放了八个处理器，代价是必须用更复杂的互连——在芯片版图上，那块交叉开关已接近一个处理器核心的面积。

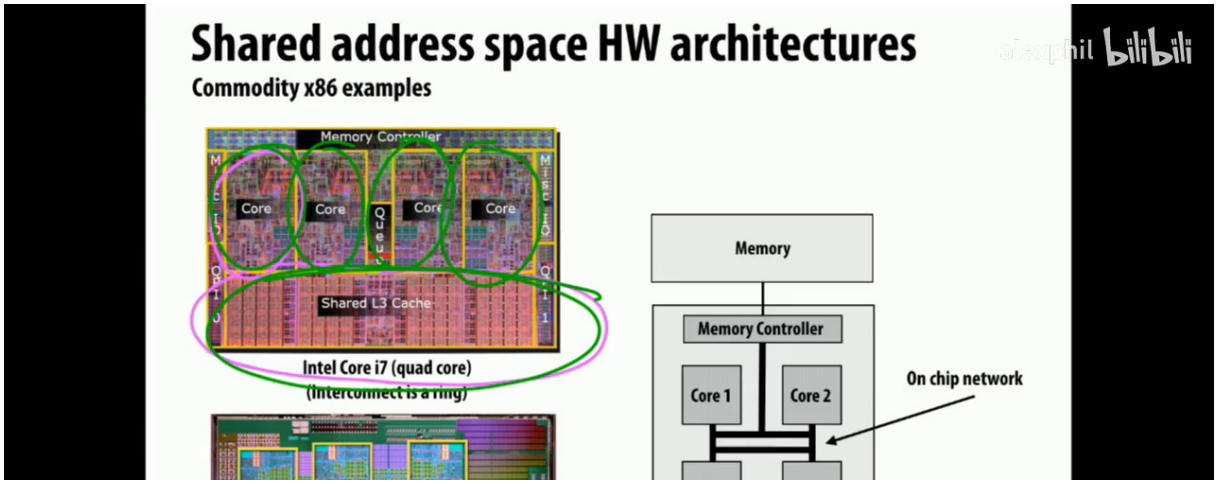


图 16: 共享地址空间的硬件实例：多核与共享缓存[16]

[15] 视频画面时间区间：00:36:00–00:36:15。若为裁剪图，时间区间仍对应原视频画面。

[16] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

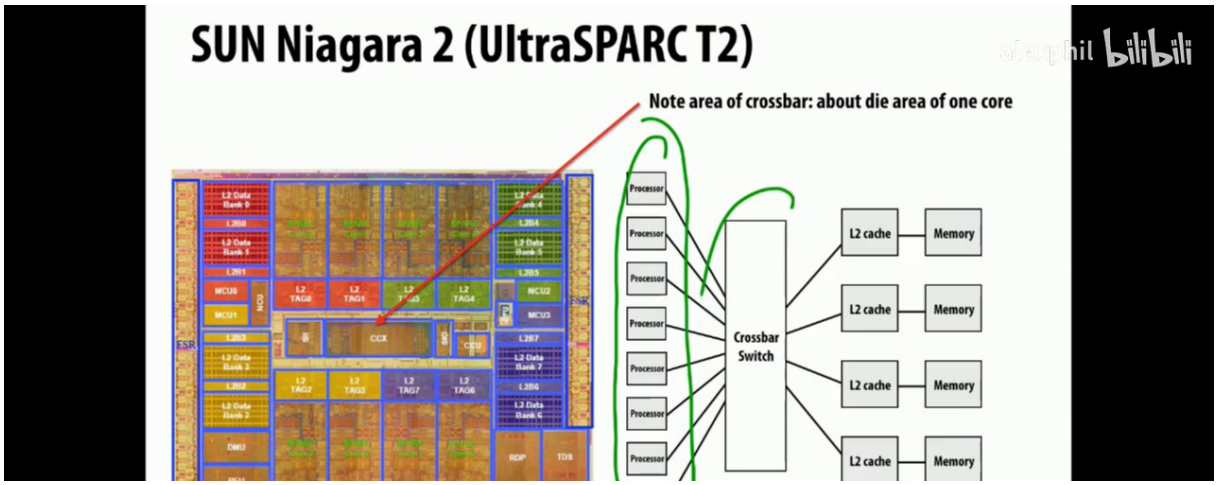


图 17: Sun Niagara 2：交叉开关的面积接近一个核心[17]

5.5 NUMA：放弃等距

要造出包含数百上千个处理器、且全部与内存等距的机器非常困难。绕开的办法是放弃等距：每个处理器配一块就近的物理内存，但仍可访问所有其他内存，只是更慢。这就是非统一内存访问 NUMA。

这个设计的收益有两个层面。私有数据（栈、代码）本来就不需要远行。更关键的是，让并行程序跑快的方式本就是划分数据，让各处理器主要处理自己的那片——讲者举的例子是把约 95% 的时间花在本地内存（视频 00:40:35）。这样既降低延迟，也减少大互连上的流量，避免拖慢他人。

代价是复杂度转移给了程序员：内存可能近而快，也可能远而慢。

NUMA 成本不会自己消失

在 NUMA 机器上忽略内存位置，程序仍然正确，只是当访问大量走远端时性能会明显下降。这类退化往往是渐进的，不会被正确性检查发现，只会在性能测量中暴露。因此在多插槽机器上做性能调优时，线程与数据的位置绑定是必须显式考虑的一项。

今天一台双插槽的笔记本或台式机就已经是 NUMA 机器。

[17] 视频画面时间区间：00:39:00–00:39:15。若为裁剪图，时间区间仍对应原视频画面。

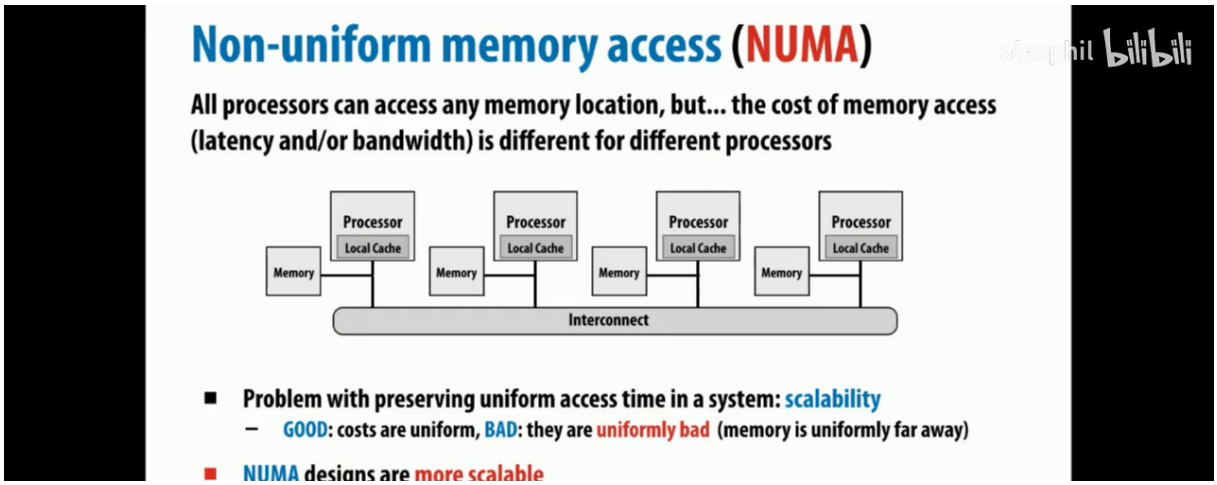


图 18: NUMA：所有处理器可访问任意内存，但远端访问更慢[18]

对称多处理为何难以做大

对称的前提是等距，而等距要求互连对所有节点一视同仁。总线的带宽不随节点数增长，交叉开关的面积则随节点数迅速膨胀。当规模上升到数百处理器时，等距的硬件成本超过收益，因此大规模机器几乎都转向 NUMA。这是被物理约束逼出的设计，不是偏好。

5.6 本章小结

同一份”共享地址空间”抽象，底层可以从共享缓存的核心群，扩展到交叉开关连接的对称多处理，再到 NUMA 的分层结构。程序员视角始终是”大家都用同一个地址空间”，但硬件代价与访问延迟随规模变化。缓存一致性则是这套抽象真正的实现难点。

6 消息传递：彻底放弃共享内存

到此为止讨论的抽象都建立在共享地址空间上。消息传递走的是相反的路：每个执行体拥有私有地址空间。

6.1 消息传递模型的通信方式

消息传递模型里，每个执行体有自己的私有地址空间，通信靠显式的 send 与 receive。消息可以带标签，接收方可以只收特定标签的消息。

这个模型最大的优势是对硬件几乎没有要求：只要能连上网络、能在机器之间传消息，代码就能跑。因此扩展到超大规模机器很容易——只需要一条快的互连，不必处理缓存一致性这些麻烦。

讲者指出，第三、第四次作业将让同一道题分别用共享地址空间和消息传递实现，学生可以自行比较偏好。

[18] 视频画面时间区间：00:39:30–00:39:45。若为裁剪图，时间区间仍对应原视频画面。

消息传递以妥协换扩展性

它要求每次通信都显式写出 send 与 receive，配对关系必须由程序员维护，这是它被认为僵硬的原因。换来的是不需要缓存一致性硬件，只要互连够快就能扩到超大规模。选择哪种模型，本质是在编程便利与机器规模之间取舍。

他的观察是，绝大多数人更偏好共享地址空间，因为消息传递在通信方式上更僵硬。

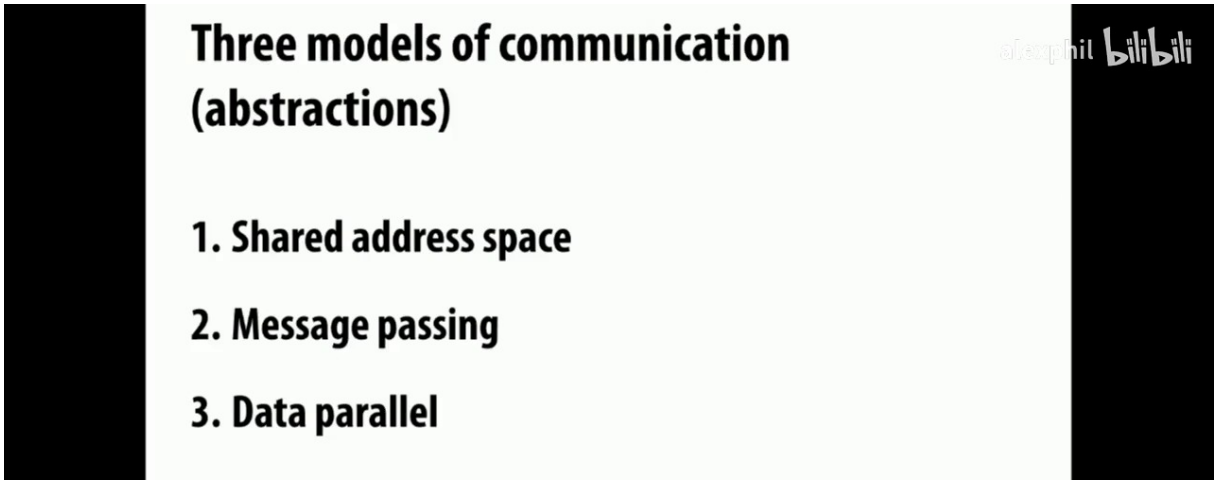


图 19: 三种通信模型：共享地址空间、消息传递、数据并行[19]

6.2 本章小结

消息传递用显式 send 与 receive 取代共享变量，代价是通信配对必须由程序员维护，收益是摆脱了缓存一致性硬件，只要互连够快就能扩到超大规模。它说明” 通信” 这件事可以用完全不同的机制表达。

另一种同样绕过共享内存的思路是把并行完全交给编译器——下一章的数据并行模型。

7 数据并行：把并行交给编译器

7.1 对全部数据应用同一函数

数据并行模型的思路是：同一份函数作用于大量数据。它在这门课上很重要，GPU 正是围绕这个模型构建的。

从 SIMD 到 SPMD 的语义放宽

早期向量机要求严格锁步：所有通道在同一时刻执行同一条指令，遇到分支只能靠掩码。SPMD 放松了这一要求，允许不同实例走不同代码路径，执行时机也不必完全一致。实现上仍常借助 SIMD，但抽象已不再等同 SIMD。

讲者做了历史对照。早期的向量超级计算机用向量指令并行，一条向量指令可以同时处理 100 多个元素，并行单位严格是” 一条指令”。今天的主流是 SPMD。今天的主流是 SPMD：对大量数据做同样的工作，但并不严格锁步，而是说” 这里是函数，对所有数据应用它”。

[19] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

7.2 forEach 只保证控制并行

这是本讲最容易踩坑的一处。讲者明确指出：forEach 严格来说只提供控制并行——它只是告诉系统循环迭代可以并行，ISPC 并不关心循环体内对数据做了什么。

因此很容易写出错误代码。讲者举了两个例子。第一个是为每个输入生成两个输出副本（取绝对值后复制两份），这段代码能编译、能正确运行，确实是一种数据并行。

forEach 不做依赖分析

把 for 换成 forEach 只声明了”迭代之间可以并行”，编译器不会验证你的迭代是否真的独立。若循环体内存在跨迭代的数据依赖（例如根据当前元素的正负去写一个由邻居决定的输出位置），多个实例可能同时写同一位置。ISPC 因为严格按向量与循环推进，结果不会真的随机，但换成调度更自由的 task 或线程，程序就会变得不确定。

第二个例子就暴露了问题：代码检查元素是否为负，然后写入输出。当相邻元素一正一负时，两个并发实例可能写同一个输出位置。讲者说从程序员角度看，程序因此变得不确定。

7.3 stream：用纯函数避免竞争

有没有更讲原则的方式利用数据并行？一个答案是 stream 模型：数据以流的形式流入，经过一个纯函数（无副作用的 kernel），产生流出。

由于 kernel 没有副作用，各元素可以任意顺序处理，也就不会出现上面那种竞争。讲者用它重写了取绝对值的例子：只看当前元素、只写对应输出，没有不确定性。

stream 模型的两个要件

流是一批可独立处理的元素；kernel 是作用于单个元素的无副作用函数。两个要件合起来才保证可任意顺序执行。只要 kernel 引入了跨元素的状态（读邻居、累加共享变量、按条件写别处），这个保证就失效。

7.4 本章小结

数据并行以交出控制权换来了简洁：forEach 声明迭代可并行，stream 模型用无副作用 kernel 保证可任意顺序执行。但原语本身不检查依赖，安全性来自 stream 的纯函数约束，而不是来自 forEach。

至此四类抽象都已交代完毕，可以回到本讲最初的判断框架。

8 总结与延伸

这一讲给出了一个可以反复使用的判断框架：看任何并行抽象，都问它在硬件上落到了哪里。

讲者的历史观察值得记录：新的抽象通常起源于有人发现了某种高效的硬件实现方式，所以最初都与硬件紧密结合。久而久之人们发现，这个抽象其实没那么依赖硬件，可以移植到别的机器上。但无论怎么移植，理解抽象与实现的差别始终是必要的，因为他强调要学会从多个不同角度看待同一件事。

ISPC 给出的最尖锐教训是：一个看起来像”多线程”的抽象，底层可以是单线程加向量指令，programCount 就是向量宽度。后三个抽象则展示了实现层次的多样化：pthreads 落在库与内核，共

享地址空间落在硬件与缓存一致性协议，消息传递只需要网络。同一份” 通信” 语义，实现在四个不同层次，代价与扩展性完全不同。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 抽象不决定性能，实现才决定。interleaved 与 blocked 表达同一个 gang 语义，在 ISPC 上却因向量加载与 gather 的差别而差距悬殊。选数据分布时必须知道目标实现。
- ” 并行语法” 不等于” 并行”。把顺序函数原样搬进 ISPC 文件只会产生冗余实例，速度不变。真正的工作在数据划分。
- 归约是并行度的收口。它必须显式表达（部分和加 reduce），且最后一步退化为标量，因此频繁的小归约是隐性开销。
- 安全的数据并行需要纯函数约束。forEach 只声明控制并行，不检查依赖；stream 模型用无副作用 kernel 补上了这一环。

本讲的边界也很清楚。讲者多次表示缓存一致性、NUMA 细节、同步原语的实现都会在后续专门的课上展开，本讲只是提出这些问题的存在。因此这里给出的是抽象的坐标系，不是任何一种模型的使用手册。

一个可以直接从本讲数据提出的下一步问题：既然 interleaved 之所以胜出是因为向量加载要求连续数据，那么当程序本身需要跨步访问（例如处理矩阵的某一列）时，是否应该改变数据布局而非访问顺序？这个问题把本讲的结论从” 选分布” 推进到” 改数据结构”，也正是后续局部性与通信那一讲要处理的。